OMARCHY QUATTRO PLUGIN ARCHITECTURE & ENGINEERING BLUEPRINT

NetLedger — System-Wide Process Lineage Bandwidth Ledger

Real-time and arbitrary multi-month network telemetry with upward process-tree attribution (`PPID` + systemd cgroup resolution), SQLite time-series rollups, and a high-signal Quickshell UI.

Plugin ID: `dev.zenmi.netledger`

Target Runtime: Quickshell / Hyprland (Arch Linux)

Storage Engine: SQLite3 Dual-Rollup Ledger

1. Problem Statement & Ecosystem Gap Analysis

SEC-01 // CONTEXT

On Arch Linux and Omarchy, network monitoring tools have historically split into two disconnected camps: interface-level historical counters (such as `vnStat`) that record months of traffic without knowing which process consumed the bytes, and live per-process monitors (such as `nethogs` and `bandwhich`) that show active sockets in real time but discard history the moment a process exits. Furthermore, when system updates or developer tools run, the kernel attributes sockets to ephemeral worker binaries like `curl`, `git-remote-https`, or `brave --type=utility` rather than the parent application the user actually launched.

Tool / Plugin	Identifier / Source	Core Capability	Architectural Limitation
Network Usage	`oliwier.network-usage` Omarchy Plugin	Packet-level per-app download/upload byte totals (TCP, UDP, QUIC, Docker).	History retention is capped at **24 hours**; no multi-week or free-form date range queries.
OmaControl	`davedes.omcontrol` Omarchy Plugin	Top-bar task manager with live resource curves and per-app historical usage.	General CPU/RAM/Net monitor; lacks a dedicated bandwidth ledger or custom date range picker.
Nexthop	`x3me/omarchy-nexthop` Omarchy Plugin	Shows live top bandwidth hog + 7d minute / 1y hourly link quality history.	Long-term history tracks ISP vs. Wi-Fi drops and speed, not cumulative bytes per process.
OpenSnitch	`opensnitch` Arch Extra Repo	Application firewall with persistent SQLite logging and GUI event filtering.	Heavy standalone Qt desktop app; does not live natively inside the Omarchy Quickshell bar.
nethogs / bandwhich	CLI Utilities Arch Official Repos	Real-time per-PID bandwidth attribution via packet capture.	Ephemeral session state only; cryptic child worker binaries are never grouped under parent apps.

2. Core Architectural Pillars of NetLedger

SEC-02 // OBJECTIVES

Four Non-Negotiable Engineering Goals for `dev.zenmi.netledger`

- **Upward Process Lineage Attribution:** Automatically trace obscure network worker PIDs (`curl`, `git-remote-https`, `alpm`, `chromium-helper`) up the `/proc/<pid>/stat` parent chain and systemd cgroup slice so traffic is grouped under human-meaningful parents like *System Update (yay / pacman)* or *Brave Browser*, while preserving full child breakdown on click.
- **Instant Free-Form & Preset Time Ranges:** Support one-click presets (`LIVE`, `24H`, `2D`, `7D`, `30D`) alongside arbitrary user-defined calendar ranges (e.g., `2026-10-01 00:00` to `2026-10-17 12:00`) or relative expressions (`-36h`, `-14d`) with sub-3ms query latency.
- **Strict Separation of Privilege (Omarchy Quattro Compliant):** Keep the Quickshell QML frontend 100% unprivileged and non-blocking by delegating packet accounting to a lightweight systemd service (`netledgerd`) that writes to a world-readable SQLite WAL database (`0644`).

3. Upward Process Lineage & Cgroup Resolution Engine

SEC-03 // ATTRIBUTION

Walking `/proc/<pid>/stat` (`ppid`) alone is insufficient on modern Linux desktops because short-lived worker processes can terminate rapidly and daemons reparent to PID 1. NetLedger combines **systemd cgroup slice inspection** (`/proc/<pid>/cgroup`) with **upward PPID ancestry traversal** to map every active socket into a 3-tier hierarchy: **Root Group** → **Parent Application/Service** → **Leaf Worker Binary**.

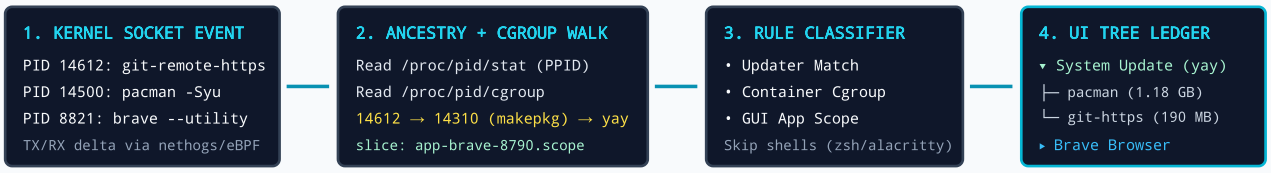


Figure 3.1 — NetLedger 4-Stage Upward Lineage Attribution Pipeline from raw socket PID to grouped UI tree.

How Concrete Real-World Scenarios Resolve

- **System Updates (`pacman` / `yay` / `paru` / `flatpak`):** When the user runs `yay -Syu` , the kernel sees separate sockets on `pacman` , `curl` , and `git-remote-https` . The lineage resolver walks up from `git-remote-https` (PID 14612) → `git` → `makepkg` → `yay` (PID 14202). Because `yay` is in the `UPDATE_MANAGERS` set, the collapsed UI displays **System Update (yay) — 1.42 GB Down / 18 MB Up**. Clicking that row expands its children: `pacman` (1.18 GB), `git-remote-https` (190 MB), and `curl` (50 MB).
- **Browsers & Multi-Process Apps (Brave / Chromium / Electron):** Network sockets live inside sandboxed utility processes (`brave --type=utility --utility-sub-type=network.mojom.NetworkService`). Walking upward and inspecting `app-brave-*.scope` groups all renderer and utility traffic under **Brave Browser** while keeping individual helper PIDs inspectable.
- **Dev Runtimes in Terminals (`bun` , `cargo` , `docker`):** When `alacritty → zsh → bun install` runs, the resolver skips transparent shells/terminals (`SHELLS_AND_TERMINALS`) so the traffic is attributed to **Bun Runtime** rather than blaming `alacritty` for every CLI download.

4. Two-Tier Storage Architecture & SQLite Rollup Schema

SEC-04 // STORAGE

To pass Omarchy's plugin validator (no `sudo` prompts inside Quickshell and zero UI thread blocking), the privileged collector daemon writes to `/var/lib/netledger/ledger.db` in WAL mode. By maintaining a two-tier rollup schema (5-minute buckets for 14 days; 1-hour buckets for 1+ year), any arbitrary multi-month query scans fewer than 2,000 rows and returns in under 3 milliseconds.

```
-- /var/lib/netledger/ledger.db (Permissions: 0644, SQLite WAL Mode Enabled)
PRAGMA journal_mode = WAL;
PRAGMA synchronous = NORMAL;

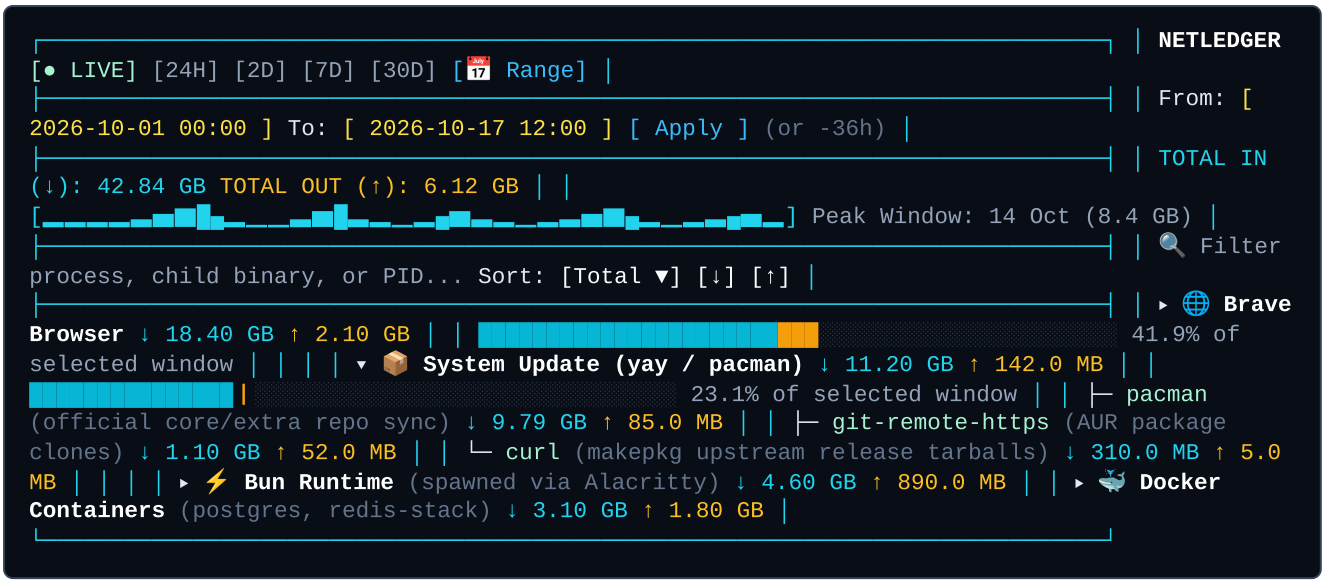
-- Tier 1: High-resolution 5-minute buckets (Retained for 14 days; drives 24H / 2D / 7D
views)
CREATE TABLE IF NOT EXISTS rollup_5min (
    bucket_ts     INTEGER NOT NULL, -- Unix epoch floored to 300s (int(now // 300) * 300)
    root_group    TEXT NOT NULL,    -- e.g., 'System Update (yay)', 'Brave Browser',
'Docker'
    category      TEXT NOT NULL,    -- 'update' | 'app' | 'container' | 'cli' | 'system'
    child_comm    TEXT NOT NULL,    -- e.g., 'pacman', 'git-remote-https', 'curl'
    child_detail  TEXT NOT NULL,    -- Sanitized cmdline context or target scope
    rx_bytes      INTEGER DEFAULT 0,
    tx_bytes      INTEGER DEFAULT 0,
    PRIMARY KEY (bucket_ts, root_group, child_comm, child_detail)
);

-- Tier 2: Long-term 1-hour buckets (Retained for 400+ days; drives 30D & custom multi-month
```

5. UI / UX Specification: Technical, High-Signal & Ergonomic

SEC-05 // INTERFACE

NetLedger's interface is engineered around **progressive disclosure**: at a glance, the top bar pill shows real-time link saturation and the icon of the #1 active process. Clicking the bar pill opens `Panel.qml`, which organizes historical and live telemetry into four high-contrast zones without visual clutter.



Signature Interaction & Ergonomic Details

- **Top-Bar Pill (`BarWidget.qml`):** Compact 85px pill displaying stacked live rates (`↓ 4.2 MB/s` in cyan, `↑ 310 KB/s` in amber) alongside a dynamic badge of the #1 active process when throughput exceeds `100 KB/s`.
- **Dual-Segment Proportional Bars:** Each row's progress bar splits horizontally into Cyan (Download share) and Amber (Upload share) normalized against the top group, immediately exposing background uploaders vs. heavy downloaders.
- **Interactive Time-Brush Histogram:** Clicking any bar in the top sparkline isolates that specific 5-minute, 1-hour, or 1-day bucket so you can answer *"What caused that 8.4 GB spike on October 14?"* in a single click.
- **Preset + Free-Form Query Bar:** Supports 1-click toggles (`LIVE`, `24H`, `2D`, `7D`, `30D`) plus a collapsible custom range bar accepting ISO dates (`2026-10-01`) or shorthand offsets (`-48h`, `-3w`, `this-month`).

6. Repository Structure & Omarchy Quattro Contract

SEC-06 // PACKAGING

File Path	Layer	Responsibility & Omarchy Validator Compliance
`manifest.json`	`Quattro Contract`	Declares plugin ID (`dev.zenmi.netledger`), entrypoints (`BarWidget.qml`, `Panel.qml`), permissions, and settings schema.
`BarWidget.qml`	`Quickshell UI`	Renders live throughput pill in the Omarchy status bar; polls live socket ring buffer every 2 seconds without blocking.
`Panel.qml`	`Quickshell UI`	Renders date-range presets, custom calendar bar, histogram, search filter, and expandable parent-child process tree.
`Model.js`	`Pure Logic`	Stateless JS module that parses `sqlite3 --json` output, formats byte units, computes bar ratios, and sanitizes strings.
`daemon/netledgerd.py`	`Systemd Service`	Privileged background collector running `nethogs -t -v 2` (or eBPF), resolving PID lineage, and writing SQLite rollups.

7. Reference Implementation: Daemon Lineage Resolver

SEC-07 // DAEMON CODE

Below is the core lineage resolution module (`daemon/netledgerd.py`) that intercepts per-PID byte deltas and walks upward through `/proc/<pid>/stat` and `/proc/<pid>/cgroup` before committing to SQLite:

```
from pathlib import Path

SHELLS_AND_TERMS = {"bash", "zsh", "fish", "sh", "sudo", "su", "alacritty", "kitty", "foot",
"wezterm", "tmux"}
UPDATE_MANAGERS  = {"pacman", "yay", "paru", "makepkg", "fwupdmgr", "flatpak", "rustup"}
SESSION_ROOTS    = {"systemd", "Hyprland", "uwsm", "quickshell"}

def read_proc_node(pid: int) -> dict | None:
    try:
        p = Path(f"/proc/{pid}")
        comm = (p / "comm").read_text().strip()
        cmdline = (p / "cmdline").read_text().replace("\x00", " ").strip()
        stat_tail = (p / "stat").read_text().rsplit(")", 1)[1].split()
        ppid = int(stat_tail[1])
        cgroup = (p / "cgroup").read_text().strip()
        return {"pid": pid, "ppid": ppid, "comm": comm, "cmdline": cmdline, "cgroup":
cgroup}
    except Exception:
        return None

def resolve_root_group(pid: int) -> tuple[str, str, str, str]:
    """Returns (root_group, category, child_comm, child_detail)"""
    chain, curr, seen = [], pid, set()
    while curr > 1 and curr not in seen:
        seen.add(curr)
        node = read_proc_node(curr)
        if not node: break
        chain.append(node)
        curr = node["ppid"]

    if not chain:
        return ("System / Kernel", "system", f"pid-{pid}", "Ephemeral socket")
    leaf = chain[0]

    # 1. Attribute all child workers (curl, git-remote-https, gpg) to parent System Updater
    for anc in reversed(chain):
        if anc["comm"] in UPDATE_MANAGERS:
            return (f"System Update ({anc['comm']})", "update", leaf["comm"],
leaf["cmdline"][:70])

    # 2. Attribute Docker / containerd cgroup slices
    if "docker" in leaf["cgroup"] or "containerd" in leaf["cgroup"]:
        return ("Docker Containers", "container", leaf["comm"], leaf["cmdline"][:70])

    # 3. Walk top-down past session managers and shells to find primary GUI App or Dev
Runtime
    for anc in reversed(chain):
        c = anc["comm"]
        if c not in SESSION_ROOTS and c not in SHELLS_AND_TERMS:
            detail = leaf["cmdline"][:70] if leaf["comm"] != c else f"{c} (main/service)"
            return (c, "app", leaf["comm"], detail)

    return (leaf["comm"], "cli", leaf["comm"], leaf["cmdline"][:70] or leaf["comm"])
```

8. Free-Form SQL Query Engine & Plugin Manifest

SEC-08 // INTEGRATION

```
// manifest.json (Omarchy Quattro)
```

```
-- Parameterized Tree Query (sqlite3 --
```